

État des lieux de xfoot

Un audit de bout en bout du SaaS de pronostics football — API, frontend, pipeline ML, app mobile et sécurité — pour savoir précisément où on en est et par quoi commencer.

Périmètre — api/, frontend-design/, mobile/, pipeline ML racine

Méthode — lecture de code, pas d'exécution destructive

BACKEND API Solide SAIN	FRONTEND À corriger XSS	PIPELINE ML Mature TESTÉ	APP MOBILE Bloquée DÉCISION	DONNÉES LIVE Dégradées SUSPENDU
--------------------------------------	--------------------------------------	---------------------------------------	--	--

Les deux blocages qui touchent tout le reste

Le compte API-Football est suspendu. Concrètement : les scores en direct, le calendrier réel de la semaine et le rapprochement quotidien des résultats tournent tous en mode dégradé (données de secours codées en dur, sans logos). Rien à corriger côté code — il faut réactiver/vérifier la facturation sur `dashboard.api-football.com`.

Le paiement in-app pour Android n'est pas tranché. `vip.html` / `billing.html` renvoient vers un paiement Chariow hors app, ce que Google Play refuse pour du contenu numérique par abonnement. Bloque toute soumission au Play Store tant que ce n'est pas décidé.

Backend — API FastAPI

api/ • FastAPI + SQLAlchemy + Postgres

L'architecture est cohérente et bien pensée : authentification JWT, facturation par abonnement pilotée par webhooks Chariow, et un moteur de prédiction (Dixon-Coles + XGBoost en ensemble) doublé d'un système de versioning de modèles maison, l'« AI Arena ».

Structure

`auth/` (inscription, connexion, session) et `billing/` (paiement, licences, webhook) sont proprement isolés dans leurs propres routeurs. `ai/arena/` (11 fichiers, ~4 800 lignes) gère le versioning des modèles, la promotion, le mode « shadow » et la supervision — bien découpé par responsabilité. À l'inverse, `main.py` fait **2 061 lignes** et héberge encore une vingtaine de routes directement (prédictions, scores live, calendrier, tout le bloc `/models/*`) au lieu d'être éclaté en routeurs dédiés comme `auth` et `billing`.

Modèle de données

Le schéma tourne autour de `ModelVersion`, qui relie notations d'équipes, prédictions et événements de promotion — une conception d'audit cohérente. Côté métier : `User`, `Subscription` (1:1, avec licence Chariow et expiration), `Match` / `MatchStats`, `PredictionLog` (rapproché avec les résultats réels par le cron quotidien).

Couverture de tests

26 fichiers de tests, environ 7 900 lignes. La partie ML/Arena est testée en profondeur (ensemble, promotion, mode shadow, anti-fuite de données, backtesting). Point faible : aucun test dédié à la liste blanche des admins (`ADMIN_EMAILS`) qui vérifie qu'un non-admin se fait bien refuser l'accès — et `main.py` étant si volumineux, la correspondance route ↔ test n'est pas évidente à l'œil.

Sécurité — ce qui est bien fait

BON

Clé JWT protégée en production

Le démarrage échoue volontairement (`RuntimeError`) si `ENV=production` tourne encore avec la clé de développement par défaut — un vrai garde-fou.

BON

Webhooks Chariow correctement vérifiés

Signature `x-chariow-signature` validée avant tout traitement, et déduplication des livraisons via une table dédiée — implémentation propre.

BON

Aucune injection SQL, aucun secret trouvé en clair

Tout passe par l'ORM SQLAlchemy ; aucune concaténation de chaînes SQL, aucun secret codé en dur détecté dans les fichiers audités.

À NOTER

Droits admin par simple liste d'emails

`ADMIN_EMAILS` est vérifié à chaque requête, sans colonne dédiée en base — pragmatique pour une petite équipe, mais à faire évoluer si l'équipe grandit.

Déploiement

Railway, déploiement automatique à chaque `git push` sur `main`, migrations Alembic exécutées à chaque démarrage (`Procfile`). Rien de manuel côté backend une fois le code poussé.

Frontend — pages statiques

frontend-design/ · HTML/CSS/JS, sans build

7 pages (~4 260 lignes au total), aucune dépendance, aucun outil de build — volontairement simple. Le point le plus sérieux à corriger est un ensemble de failles XSS liées à l'usage systématique de `innerHTML` sans échappement.

Faillles à corriger en priorité

CRITIQUE

Injection possible via `innerHTML` non échappé

Noms d'équipe, URLs de logo et messages d'erreur de l'API sont injectés tels quels dans le HTML sur `index.html`, `live.html`, `history.html` et `arena.html` — y compris dans un attribut `onerror` sur `index.html:245-248`, le point le plus exploitable (une URL de logo mal formée pourrait casser hors de l'attribut et exécuter du JS). Aucune fonction `escapeHtml()` n'existe dans le projet.

IMPORTANT

Jeton JWT stocké en `localStorage`

Lisible par tout script injecté — combiné aux failles XSS ci-dessus, un vol de session devient possible. Corriger le XSS réduit déjà l'essentiel du risque ; migrer vers un cookie `httpOnly` serait l'étape suivante, plus lourde côté backend.

Duplication à factoriser

Rien n'est partagé au-delà de `api.js` (jeton, appels API, garde d'authentification). Trois implémentations différentes de `getInitials()` coexistent — celle de `live.html:91-93` plante si le nom de l'équipe est vide, contrairement aux deux autres qui s'en protègent. L'affichage du nom d'utilisateur est copié-collé à l'identique 6 fois. Aucun fichier `components.js` ne mutualise ces bouts de logique.

Le point chaud : `arena.html`

1 165 lignes — plus du double de la page suivante la plus grosse (`index.html` , 518 lignes). Mélange interface d'administration des modèles, prédiction par ensemble et historique de promotion dans un seul fichier, avec sa propre logique dupliquée d'authentification et d'affichage. Premier candidat à une découpe en plusieurs fichiers.

Cohérence

Bon point : les 7 pages affichent l'identité utilisateur de façon cohérente (nom, repli sur l'email) — aucune page n'a été oubliée lors du dernier changement. Petit détail : le cache-busting CSS (`?v=5` à `?v=7` selon la page) n'est pas synchronisé, risque de feuille de style obsolète en cache sur certaines pages.

Pipeline ML & opérations

racine du repo · scripts + crons Railway

C'est la partie la plus mature du projet : pipeline de bout en bout (données brutes → features → modèle entraîné → artefact validé → modèle servi), doublée d'un système de production plus sophistiqué (« Al Arena ») avec mode shadow et promotion de modèles auditée.

Les 4 tâches planifiées (Railway)

HORAIRE (UTC)	SCRIPT	RÔLE
Lun 04:00	refresh_and_retrain.py	Réentraînement hebdomadaire complet
Quotidien 05:30	generate_live_predictions.py	Génère les prédictions ACTIVE + SHADOW
Quotidien 06:30	fetch_daily_results.py	Rapproche les prédictions des résultats réels
Quotidien 07:00	evaluate_live_models.py	Évalue la performance, nourrit les décisions de promotion

L'ordre est pensé : prédictions → résultats → évaluation, espacés d'environ une heure. Tant que le compte API-Football reste suspendu, la tâche de 06:30 ne peut rien rapprocher : les prédictions logguées s'accumulent en statut `pending` sans être validées.

Historique récent (phases 10 & 11)

- **Phase 10** — validation live scopée par version de modèle, API de promotion protégée admin, table d'audit des promotions.

- **Phase 11** — comparaison stricte ACTIVE vs SHADOW sur les mêmes matchs, nouveaux endpoints publics `/models/shadow/*`.

Hygiène du dépôt

BON

Zéro fichier binaire traîné par erreur

`.gitignore` exclut correctement `*.db`, `__pycache__`, `.env` — confirmé par `git ls-files`, aucun `.db` / `.pkl` / `.zip` suivi dans tout le dépôt.

À NOTER

Données d'entraînement versionnées en clair dans git

Les CSV de features (jusqu'à 2,18 Mo) sont commités directement plutôt que stockés à part — acceptable à cette taille, mais à surveiller si le volume de données grandit.

App mobile (Android)

mobile/ · Capacitor autour de frontend-design/

Le wrapper Capacitor empaquette le site existant sans le dupliquer — bon choix, aucune divergence de code à maintenir entre le web et le mobile. iOS repoussé faute de Mac, structuré pour être ajouté plus tard.

Reste à faire, uniquement à la main

- Installer Android Studio + JDK (absents de cet environnement) pour compiler et tester — rien n'a encore tourné, pas même sur émulateur.
- Ajouter `https://app.xfoot.site` aux `ALLOWED_ORIGINS` Railway — sinon tous les appels API échouent silencieusement depuis l'app.
- Remplacer l'icône et le splash screen par défaut.
- Générer et **sauvegarder** le keystore de signature (`.jks`) — irrécupérable en cas de perte.
- Créer le compte développeur Google Play (25 \$) et la fiche Play Console, avec URL de politique de confidentialité.
- Trancher la question du paiement in-app (voir encadré en haut de page) avant toute soumission.

Par quoi commencer

ordre recommandé

1

Corriger les échappements `innerHTML` côté frontend

C'est la seule faille de sécurité réellement exploitable trouvée dans l'audit. Une fonction `escapeHtml()` partagée, appliquée partout où une donnée API atterrit dans du HTML, referme le risque XSS et, avec lui, le risque de vol de jeton JWT.

2

Réactiver le compte API-Football

Rien à coder — juste vérifier la facturation sur le dashboard. Tant que ce n'est pas fait, scores live, calendrier réel et rapprochement des résultats restent en mode dégradé.

3**Trancher la question du paiement in-app Android**

Bloque toute soumission au Play Store. Décision à prendre avant d'investir plus de temps sur le mobile : intégrer Google Play Billing, ou rendre l'app muette côté paiement (renvoi vers le site).

4**Découper `main.py` et `arena.html`**

Les deux plus gros fichiers du projet (2 061 et 1 165 lignes) mélangent plusieurs responsabilités. Pas urgent, mais chaque nouvelle fonctionnalité y sera plus coûteuse à ajouter tant que ce n'est pas fait.

Analyse réalisée par lecture de code (aucune modification) – `api/`, `frontend-design/`, `mobile/`, pipeline ML racine.